Overview

This document outlines the Grafana-based monitoring system implemented for the BrainBytes AI Tutoring Platform, covering key dashboards, metrics, and alert configurations. It is intended for use by developers, DevOps engineers, QA, and product managers to monitor system performance, identify bottlenecks, and support scaling

DASHBOARD CATALOG

1. BrainBytes System Overview Dashboard

Purpose:

Provides a high-level view of the system's health, focusing on AI response time and real-time user activity.

Key Metrics & Visualizations:

- AI Response Time (brainbytes_ai_response_time_seconds): Line graph tracking average AI response time.

- Active Sessions (brainbytes_active_sessions): Line graph of concurrent tutoring sessions.

Target Audience:

- DevOps Engineers
- Backend Developers
- Product Managers

Use Cases:

- Detect AI slowdowns or outages
- Monitor session trends for scaling
- Identify periods of high/low usage

2. Error Analysis Dashboard

Purpose:

Analyze error patterns by endpoint and correlate with system resources.

Key Metrics & Visualizations:

- Error Distribution by Endpoint
- Error Distribution by Status Code
- Recent Errors Table
- CPU vs Error Rate Correlation

Target Audience:

- Backend/API Developers
- QA Engineers
- DevOps Teams

Use Cases:

- Debug and trace root causes
- Detect deployment-related failures
- Monitor for unusual error spikes

3. Resource Optimization Dashboard

Purpose:

Monitor and improve resource usage across containers and services.

Key Metrics & Visualizations:

- Container Memory Usage
- Total Network Out (All Containers)
- CPU per Request Efficiency
- Resource Usage vs Request Volume

Target Audience:

- Infrastructure Engineers
- System Architects
- Performance Engineers

Use Cases:

- Identify inefficient services

- Plan for horizontal scaling
- Detect resource exhaustion

METRIC DICTIONARY

Metric Name	Description	How It's Calculated	Normal Value Range	What Unusual Values Indicate
brainbytes_http_requests_total	Total number of HTTP requests	Incremented by 1 for each HTTP request, labeled by method, endpoint, and status	Increases steadily	Sudden spikes: traffic surge; drops: service outage
brainbytes_http_request_duration_seconds	HTTP request duration in seconds (histogram)	Observed for each request, labeled by method, endpoint, and status	0.01–2 s typical	High: backend slowness, resource issues
brainbytes_active_sessions	Number of active tutoring sessions	Gauge set by backend logic or user activity	0–20+	Sudden drop: user disconnects; spike: load test or bug

brainbytes_messages_per_subject_total	Total messages per subject	Counter incremented for each message, labeled by subject	Increases with use	Drop: inactivity; spike: spam or test
brainbytes_mobile_response_time_seconds	Mobile client response time (histogram)	Observed for each mobile request	0.2–2s typical	High: mobile network issues, backend slowness
brainbytes_data_usage_bytes_total	Total data usage in bytes (sent/received, by client type)	Counter incremented by estimated bytes per request/response	Increases with use	Spike: large payloads, downloads, or data leak
brainbytes_intermittent_connectivity_events	Current number of detected intermittent connectivity events	Gauge incremented when a simulated user goes offline	0 (normal)	High: network instability, user disconnects
brainbytes_ai_response_time_seconds	AI response time in seconds (histogram)	Observed for each AI response	0.2–1s typical	High: AI/model slowness, backend bottleneck

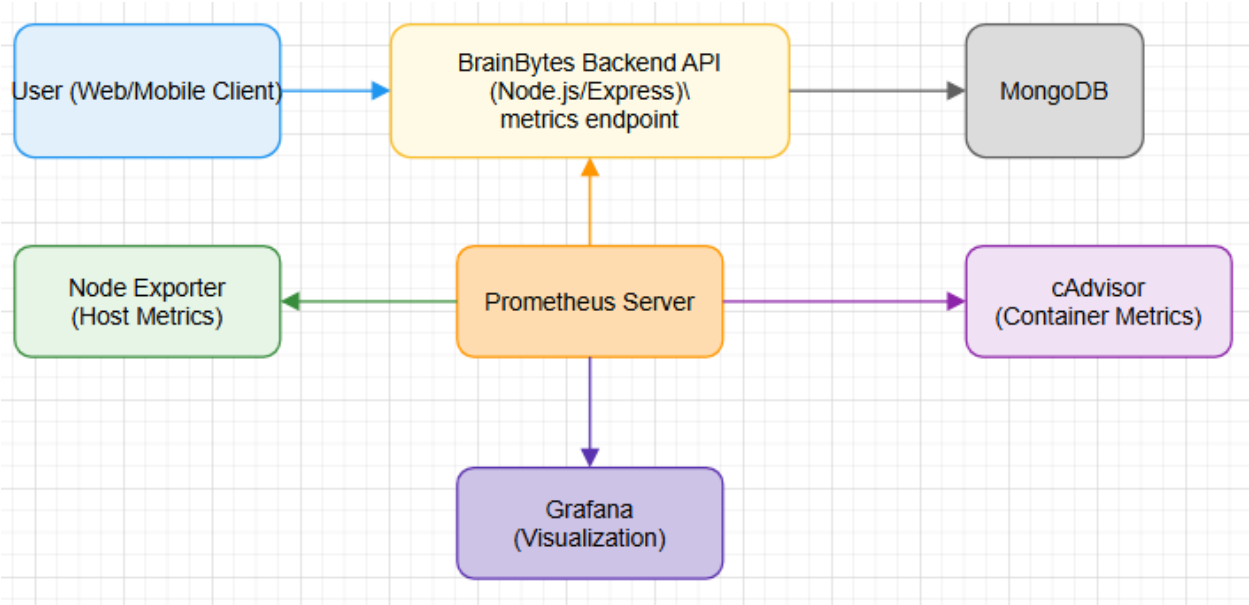
brainbytes_daily_active_users	Unique user emails who sent a message in last 24 hours	Gauge set by counting unique userEmails in DB in last 24h	1–20+	Drop: user churn; spike: new users or test

ALERT REFERENCE GUIDE

Alert Name	Metric/Panel	Threshold & Criteria	Severity	Response Procedure
AI Response Time Degradation	brainbytes_ai_response_time_seconds	IS ABOVE 2s for 2m	Critical	1. Check backend/AI logs 2. Scale up resources if needed 3. Investigate model or infra slowness 4. Communicate status to team
Error Rate High	Error Distribution by Status Code	IS ABOVE 0.1 (10%) for 2m	Critical	1. Investigate error logs and recent deployments 2. Roll back changes if needed 3. Communicate status to team

MONITORING ARCHITECTURE

Data Flows



User (Web/Mobile Client) → Backend API:

Users interact with the BrainBytes Backend API by sending requests (e.g., messages, queries).

Backend API → MongoDB:

The backend stores and retrieves application data from MongoDB.

Prometheus Server ← Backend API:

Prometheus scrapes the /metrics endpoint on the backend API at regular intervals to collect application-level metrics (such as request rates, errors, and custom metrics).

Prometheus Server ← Node Exporter:

Prometheus scrapes Node Exporter to collect host-level metrics (CPU, memory, disk usage, etc.).

Prometheus Server ← cAdvisor:

Prometheus scrapes cAdvisor to collect container-level metrics (resource usage, performance, etc.).

Grafana ← Prometheus Server:

Grafana queries Prometheus to visualize metrics and display dashboards and alerts for system health, performance, and errors.

Retention Policies and Performance Considerations

Prometheus Retention Policy:

By default, Prometheus retains metrics data for 15 days. This can be configured using the --storage.tsdb.retention.time flag. For long-term storage, consider integrating remote storage solutions.

Performance Considerations:

Scrape Interval: Set scrape intervals (every 15s or 30s) based on the required granularity and storage capacity.

Resource Usage: High-frequency scraping and many metrics can increase storage and CPU usage on the Prometheus server.

Dashboard Performance: Complex Grafana dashboards with many panels or long time ranges can slow down queries. Optimize queries and use summary metrics where possible.

Scaling: For large deployments, consider federating Prometheus servers or using remote storage backends.

Security Measures

Network Access Control:

Restrict access to Prometheus and Grafana using firewalls, security groups, or network policies. Only trusted users and systems should access monitoring endpoints.

Authentication and Authorization:

Enable authentication for Grafana dashboards and Prometheus web UI.

Use role-based access control (RBAC) in Grafana to limit user permissions.

Transport Security:

Use HTTPS for Prometheus and Grafana endpoints to encrypt data in transit.

Secure the /metrics endpoint on the backend API so only Prometheus can access it (e.g., via IP allowlisting or authentication).

Regular Updates:

Keep Prometheus, Grafana, Node Exporter, and cAdvisor updated to the latest stable versions to patch security vulnerabilities.

Alerting on Anomalies:

Configure alerts for unusual activity or potential security incidents (e.g., unexpected spikes in traffic or resource usage).

DEMONSTRATION PLAN

This section outlines the recommended flow for a 10–15 minute demonstration of the BrainBytes monitoring system.

Key Dashboards to Highlight

System Overview Dashboard:

Presents AI response time and active user sessions, providing a high-level view of system health and engagement.

Error Analysis Dashboard:

Displays error rates by endpoint and status code, enabling rapid identification of issues and alerting on abnormal error patterns.

Resource Optimization Dashboard:

Tracks CPU, memory, and network usage across containers, helping to identify resource bottlenecks and optimize performance.

Demonstration Flow

Normal Operation

Begin by running the demo data generator script with default settings.

Observe stable AI response times, steady active session counts, minimal error rates, and normal resource usage.

Use this scenario to explain what healthy system behavior looks like on each dashboard.

High Load Scenario

Increase the number of simulated users in the script to generate higher traffic.

Observe spikes in active sessions and resource usage, and potential increases in AI response time.

Highlight how the dashboards dynamically reflect increased load and discuss the importance of monitoring these metrics.

Error Condition Scenario

Adjust the script to increase the error rate.

Observe error panels showing spikes in error rates and the triggering of the error alert in Grafana.

Explain the significance of timely alerting and how error visualization aids in rapid diagnosis.

Alert Triggering and Resolution

Simulate backend slowness or further increase load to trigger the AI response time alert.

Demonstrate the alert appearing in Grafana.

Restore normal conditions in the script and show how alerts resolve and dashboards return to baseline.

Explanations for Important Visualizations

For each dashboard panel, provide explanations covering:

The meaning of the metric (e.g., response time, error rate, resource usage).

Typical value ranges and what constitutes normal versus abnormal behavior.

How these visualizations support operational awareness and incident response.

This demonstration plan is designed to clearly showcase the monitoring system's capabilities, including real-time visualization, alerting, and the ability to respond to both normal and abnormal operating conditions.

DEMO DATA GENERATOR

The demo data generator script (simulate-traffic-advanced.js) is designed to produce realistic and varied traffic patterns for the BrainBytes monitoring system. It enables demonstration of all key monitoring features, including dashboards and alerting.

Script Features

Normal Operation:

Simulates typical user activity with steady request rates, low error frequency, and normal response times.

High Load Scenario:

Increases the number of concurrent simulated users to generate a surge in traffic, resulting in higher resource usage and potential increases in response time.

Error Condition Scenario:

Raises the error rate in simulated requests to trigger error alerts and visualize abnormal system behavior.

Intermittent Connectivity:

Simulates users with unstable connections, generating connectivity events for relevant metrics and panels.

How to Run the Demo Generator

Ensure all services are running:

Backend server

Prometheus

Grafana

Run the script:

In your project directory, execute:

node simulate-traffic-advanced.js

Observe dashboards:

Open Grafana and monitor the dashboards as the script generates data.

How to Customize the Demo Generator

Change the number of simulated users:

At the end of the script, find the line that starts the users (startConcurrentUsers) and change the number to your desired user count.

Adjust error rate:

In the makeRequest function, look for the line that sets the error rate (const isError = Math.random() < 0.2) and change the value to increase or decrease the error frequency.

Modify load patterns:

Edit the getLoadDelay(hour) function to change how frequently requests are sent, simulating different times of day or load profiles.

Simulate high load or error spikes:

Temporarily increase user count or error rate to trigger alerts and demonstrate system response.

Demonstration Scenarios

Normal:

Use default settings to show stable metrics and healthy system behavior.

High Load:

Increase user count to observe spikes in sessions, resource usage, and response times.

Error Spike:

Raise error rate to trigger error alerts and visualize error handling.

This demo generator ensures all key monitoring features can be demonstrated interactively and can be easily customized for different scenarios.